

index.php file in a text editor and enter the following line:

```
<? echo "Hello World! This is server 1." ?>
```

Visiting the website http://server_1.unixclinic.net/index.php should verify the working of PHP.

Configuration of Server2

Before we set up load balancing using the reverse proxy feature of *nginx*, we need to have a second identical server. Although as an experiment, you can configure another virtual server in *nginx* listening on a different port and use it, we will use another server that is configured identically to this primary server. The following is the virtual server configuration of server2, which is a separate physical server:

```
root@server_2 # cat /etc/nginx/sites-enabled/default
server {
    listen 8001;
    server_name server_2.unixclinic.net;
    access_log /var/log/nginx/server_2.unixclinic.net-access.log;
    error_log /var/log/nginx/server_2.unixclinic.net-error.log;

    location / {
        root /var/www;
        index index.html index.htm index.php;
    }

    location ~ \.php$ {
        include /etc/nginx/fastcgi_params;
        fastcgi_pass 127.0.0.1:9000;
        fastcgi_index index.php;
        fastcgi_param SCRIPT_FILENAME /var/www/$fastcgi_script_name;
    }
}
```

The content of the *index.php* file in the document root is as follows:

```
root@server_2 # cat /var/www/index.php
<? echo "Hello World! This is server 2." ?>
```

Configuring the reverse proxy (Layer 1)

After getting the member servers ready, we will now proceed to configure *nginx* as the reverse proxy. Open the */etc/sites-available/rev-proxy-lb* file in a text editor and enter the following data:

```
upstream web_servers {
    server server_1.unixclinic.net:8001 max_fails=2 fail_timeout=30s;
    server server_2.unixclinic.net:8001 max_fails=2 fail_timeout=30s;
}

server {
    listen 80;
    server_name www.unixclinic.net;
```

Connection distribution methods

The connection distribution methods in *nginx* describe how the connection load is balanced or distributed across member servers. We have already looked at the weight parameter in detail, which distributes the connection based on the weight assigned to a member server.

Another method for connection distribution is *ip_hash*. Sometimes it is required that a client request is always transferred to a particular member server—the *ip_hash* directive can be used in such cases. When a client connects to the server, the *ip_hash* directive calculates the hash of the client's IP address and keeps a track of the member server it is connecting to. All subsequent requests from that client are passed to the same member server, if it is available.

In the case of a member server being unavailable, the requests are passed on to another member server. If a certain member server has to be taken down for some reason and for quite some time, then it is advised to mark that server as 'down'. The weight and *ip_hash* directives are incompatible with each other and hence cannot be combined together.

```
rewrite ^/(.*) http://unixclinic.net/$1 permanent;
}

server {
    listen 80;
    server_name unixclinic.net;
    access_log /var/log/nginx/rproxy_1-access.log;
    error_log /var/log/nginx/rproxy_1-error.log;

    location / {
        proxy_pass http://web_servers;
    }
}
```

The first server directive just contains a rewrite to redirect all requests coming to <http://www.unixclinic.net> to <http://unixclinic.net>.

The upstream directive is in the *nginx_http_upstream* module that balances load across multiple back-end servers. This module uses a simple round-robin load-balancing algorithm. The upstream directive specifies a set of servers that can be used in other directives such as *proxy_pass* and *fastcgi_pass*.

The server directive specifies the name of the member server and the parameters applicable for a server.

- The name part can contain a domain name, IP address, a port number and a UNIX socket. If a domain name resolves to multiple IP addresses (multiple A records in the DNS for a domain, see note), then all the IP addresses are used.
- A weight can be assigned to each server to specify its priority level for handling requests. If a weight is not assigned, then it is considered as 1. For example, if a weight of 2 is specified for *server_1.unixclinic.net* as follows, then for every three requests, two will be